Relation to Graph Concepts

Rajiv Sambasivan

October 23, 2025

1 Overview and Scope

Graphs are a common analysis tool in data science and machine learning. In some cases, the need to use graphs is recognized upfront. The data model is designed to be a graph from the start. In other cases, the need to use graphs is recognized during the analysis phase. In such cases, the data is typically in a relational format. For example, data may come from an ERP (Enterprise Resource Planning)system, a CRM (Customer Relationship Management)system or Sales system (Salesforce). The challenge is to convert the relational data into a graph structure that is suitable for analysis. This document provides an overview of the concepts related to this transformation. The scope of the document is limited to the transformation of relational data into graph structures. Some familiarity with the relational model and the entity-relationship model is assumed. On the target side, the scope of this document is limited to the construction of homogeneous, bipartite and heterogeneous graphs. The document does not cover advanced topics such as hypergraphs, temporal graphs etc. The document stops at the point of graph construction. The document does not cover graph analysis or graph learning.

2 Motivations to Use Graphs

Typically when analysts want to explore graphs as a data representation, *heterogeneity* in the data is a key motivation. Heterogeneity is a term that originates in statistics. In statistics, heterogeneity refers to the presence of sub-populations within an overall population that exhibit different characteristics. For example, in a clinical trial, patients may respond differently to a treatment based on factors such as age. A company may have different customer segments that exhibit different purchasing behaviors. Heterogeneity is a common characteristic of business data. Graphs provide a natural way to represent and analyze heterogeneous data. Graphs can capture the relationships between different entities in the data. Graphs can also capture the different types of entities in the data. This makes graphs a powerful tool for analyzing heterogeneous data.

How hetrogeneity manifests itself in data depends on the type of analysis being performed. There are three common types of data analysis: *cross-sectional,* *longitudinal* and *panel*. Each type of analysis has its own characteristics and challenges when it comes to identifying and accounting for heterogeneity in the data.

In a *cross-sectional* analysis, sources of heterogeneity may be identified through domain expertise or by applying data-driven techniques. A dataset where you are modeling housing prices over a wide geographic region at a specific point in time is an example of a *cross-sectional* dataset. A real estate domain expert may have developed ways to partition the housing market into segments where each segment has similar relationship between the house price and the characteristics of the hous. Alternatively, you can use machine learning to systematically uncover these sources. For example, you can use a Classification and Regression Tree (CART)[1] to identifiy regions that have a similar predictor-response characteristics and then develop models for each of these segments [15]. The regions identified by the regression tree are the sources of heterogeneity in the data.

In a *longitudinal analysis*, heterogeneity occurs over time. A study where you are monitoring the blood glucose of a subject over time, or, a study where you are tracking the price of a commodity like coffee over time is an example of a *longitudinal analysis*. Tracking changes with respect to time using *change point analysis* can tell you when and how many changes have occurred in the period that you are monitoring. See [14] for an example. Change point detection is a canonical time series analysis task and many methods such as the *Matrix Profile*[18] have been used to identify change points. Each segment between change points can be treated as a homogeneous segment. You can then model each segment separately. The segments identified by change point analysis are the sources of heterogeneity in the data.

In *Panel Data*, you need to account for sources of variation you have observed with both *cross-sectional* and *longitudinal* data. A company may track the purchasing behavior of a set of customers over time. In such cases, you need to account for heterogeneity across customers as well as heterogeneity over time.

2.1 The Relational Model

[3] introduced the relational model, which is the logical foundation for relational databases. In this model, data is represented as relations (tables), where each relation consists of tuples (rows) and attributes (columns). An algebraic framework for manipulating these relations is provided by relational algebra, which includes binary operations (operate on two relations) such as join and union, as well as unary operations like selection and projection (operate on a single relation). The relational model provides a mathematical framework for representing and querying data.

2.2 The Entity-Relationship Model

The entity-relationship (ER) model, introduced by [2] is a framework for describing the *semantic* data abstractions and their relationship with each other.

For example, if your use case is about how students in a university register for courses, then *students, course, registration* might be a set of data abstractions. The *registration* may connect a student with a course for a particular semester. From a *data representation* point of view, all semantic abstractions may be represented as relations.

2.3 The Entity-Relationship to Relational Schema Mapping

There exist guidelines and a body of knowledge around mapping an *Entity-Relationship Model* to a *Relational Schema*. See [5], for example. The working context for this document that you have a set of extracts from a relational database, in other words, this step has been done.

2.4 Use Case Data

Your *Use Case Data* is what you want to convert to a graph. The data for your use case is a set of extracts that correspond to the entities and their relationships. As a modeler, you are expected to know:

- The semantic abstractions relevant to your use case. These are the *entities* in your data extract. You should spend sometime to develop a clear understanding of the entities in your use case.

- The *semantic relationships* in your usecase. You should understand how *entities* in your usecase are related. You should spend sometime to understand the variations that are relevant from the perspective of your use case.

- The *semantic to extract mapping* for your usecase. You should know which extract file correspond to a particular entity or relationship. The extracts are what you get from the relational databases supplying the data for your use case. As an extension, we can look at connectors that fetch data from source relational systems, but for now we will assume you have file extracts of relations and entities. Extract to semantic mapping may be a multi-step process. For example, you may need to join two or more extracts to get the data for a particular entity. You should spend sometime to understand how the extracts map to the entities and relationships in your use case. Also you may need to transform the data in the extracts to get the data for a particular entity or relationship. You should spend sometime to understand how the extracts map to the entities and relationships in your use case.

2.5 Analysis

The use case under consideration seeks to accomplish a set of business goals. Use case implementation needs to be informed by a set of data analysis tasks.

Identifying heterogeneity in your data, as it pertains to your use case objectives, is often one of the first analysis steps. Assessment for heterogeneity is asking if all parts of your dataset are similar or homogeneous. For example, in a financial dataset, you may have data from different lines of business, different product categories, or different geographic regions. Each of these segments may have different characteristics that need to be accounted for in your analysis. For example, if your use case forecasts revenue or demand for a product or service for a particular line of business, the variables that go into the model and statistical characteristics of the quantity you are trying to model may be different for each line of business. Heterogeneity is a fact of life in business data. If there is no heterogeneity then *Independent and Identically Distributed (IID)* assumptions may be valid and you can use standard statistical and machine learning methods. Heterogeneity identification is one of the tasks that you will typically need to perform for analytics and machine learning tasks with business data. The level of dilligence and rigor to identify heterogeneity falls in a spectrum, from simple visual inspection of histograms and densities of the variables of interest to formal *Analysis of Variance* tests on hypothesis about these variables. At a minimum, you will want to do basic visual inspection of histograms and perhaps simple cluster analysis to understand the distribution of variables in your dataset.

Usually a use case will have multiple performance evaluation metrics. You will typically need to understand how your data shapes these metrics. The repository includes an analysis of loan performance data from the Small Business Administration (SBA). In this use case, there are two performance metrics of interest: *recall* and *precision.* Please see this document for details. The analysis of the data is done from the perspective of these two metrics.

Sometimes design choices that are optimal for one metric may adversely affect another important metric, trade offs and prioritization are typically involved in these situations. For example, in the SBA loan performance use case, increasing recall may come at the cost of decreasing precision. What is acceptable for your use case. This should be done in collaboration with domain experts. Clearly, the above discussion is high level. The details of the analysis will depend on the use case.

Attribute selection and feature engineering are also important analysis tasks. The details of these tasks will depend on the use case. At a minimum, you will want to do the following:

- Feature Selection: Identify the attributes that are relevant to your use case. This can be done using domain knowledge or by using statistical methods such as correlation analysis or mutual information.

- Categorical Variable Analysis: If you have categorical variables, you will need to understand the level of support for each category. You may need to group categories with low support into another category or drop them from the dataset.

- Missing Value Analysis: You will need to understand the extent of missing values in your dataset. You may need to impute missing values or drop records with missing values.

- Categorical Variable Encoding: You will need to encode categorical variables into a format that can be used by machine learning algorithms. This can be done using various encoding techniques such as one-hot encoding, label encoding, target encoding etc.
- Numeric Variable Transformation: You may need to transform numeric variables to improve their distribution.
- Dimensionality Reduction: If you have a large number of features, you may need to reduce the dimensionality of your dataset. This can be done using techniques such as Principal Component Analysis (PCA) or a manifold learning technique.
- Attribute Creation: You may need to create new attributes from existing attributes. For example, you may want to create a new attribute that is the ratio of two existing attributes. In some cases, you may want to create interaction terms between two or more attributes.

Usually, you don't get to your desired level of analysis in one step. In supervised problems, you will need to consider different modeling approaches, usaually starting with simple models and then moving to more complex models. Understanding the relationship between features and the model target is a critical part of the analysis. This will drive your decision about the type of model you want to use. This is where concepts like *bias-variance tradeoff* come into play. You will need to understand the tradeoff between model complexity and model performance. You will also need to understand the tradeoff between model interpretability and model performance. This is typically done in collaboration with domain experts.

In unsupervised problems, you will need to consider different modeling approaches to arrive at a meaningful representation of your data. For example there are no simple answers to the following questions:

- How many clusters are there in your data?
- What is the right distance metric to use for clustering?
- What is the right dimensionality reduction technique to use for your data?
- What is the right number of dimensions to use for your data?
- Is a Euclidean assumption valid for your data? Do I need to consider a manifold learning technique?

Of course, some questions may be relevant to both supervised and unsupervised problems. For example, *Is a Euclidean assumption valid for your data? Do I need to consider a manifold learning technique?* is relevant to both supervised and unsupervised problems. Is the data sparse or dense? is another question that is relevant to both supervised and unsupervised problems. Sparse data may require different modeling approaches compared to dense data.

This is where descriptive analytics is really useful. By taking the time to understand the critical questions about your data, you will be able to make informed decisions about the type of model you want to use.

2.6 Model Building and Operationalization

When analysis is complete, we have a plan for how data is going to be processed and fed as input to models. Interpretation and presentation of model outputs is also determined in the analysis step. Operationalizing the model and monitoring the model is the next step. This is typically done by an MLOps team. This is outside the scope of this document.

3 Guidelines for Implementation

The implementation consists of an initial assessment of the data, analysis for heterogeneity and a graph mapping. The guidelines for performing these steps are given below.

3.1 Initial Assessment

The initial assessment is done on the raw data obtained from the source systems. For the initial assessment, use a `jupyter notebook` to do the following:

- Inspection of Dataset: This is a quality assessment of the extracts you received for the entities and the relations. For each entity, verify that you have the required attributes in the extract. Make note of the desired attributes, you will need this to define the workflow to process the dataset.
- A *critical* point to note and understand at this point is the nature of analysis the use case is performing, i.e., are you doing a *Cross-Sectional, Longitudinal or Panel* study.
- Be clear about your perspective on heterogeneity in the data. You are typically in one of three situations:
  1. You have domain expertise about sources of heterogeneity in the data. You don't need to do any data-driven analysis to identify sources of heterogeneity.
  2. You don't have domain expertise about sources of heterogeneity in the data. You need to do data-driven analysis to identify sources of heterogeneity. You need to get this validated by domain experts.
  3. You believe that business conditions have changed and you need to identify new sources of heterogeneity in the data. You need to do data-driven analysis to identify sources of heterogeneity. You need to get this validated by domain experts.

- Understand and document the business rules that must be satisfied in your final data representation.For example, if you have a finish time and a start time, you may want to verify that the finish time is no earlier than the start time. On financial data, signs, value-ranges etc., may need verification. Make note of how the business rules must be verified and how non-conforming records can be flagged, logged and dropped from the dataset.

- If you need numeric features for heterogeneity analysis, then you might want to featurize the dataset. If, for example you need to use clustering to group data into similar subsets, then you need to perform featurization where data for each entity chain is completely numeric. If you want to rely on segmentation, which translates to a group-by operation, to identify heterogeneity, you don't need to featurize at this point. So when you want to featurize a dataset will depend on how you want to do heterogeneity analysis.

The note comments in each of the steps above will be used to guide further processing. A resource that is old but still relevant is [12]. The book provides a good overview of data preparation tasks.

3.2 Heterogeneity Analysis

3.3 Graph Mapping

The challenge you are likely to face is the decision about how to map the source entities and relationships to a graph structure. There are three common analysis themes relating to graph structure that we frequently encounter in enterprise applications. The first theme corresponds to a *homogeneous graph*. In such a graph, the nodes are the same type of entity. In these graphs there is a *pivotal entity* that is the focus of the analysis. For example, in a financial use case, the pivotal entity may be a *customer*. In a health care use case, the pivotal entity may be a *patient*. In a manufacturing use case, the pivotal entity may be a *machine*. The analysis task is to understand the behavior of the pivotal entity. Understanding the behaviour of the pivotal entity is a major application of graph analysis in enterprise use cases. Please see [6] and [7] for a detailed discussion. When you want to connect two entities from the perspective of identifying heterogeneity in the relationship between the two entities (normal and anomalous behavior can be regarded as one example of this), then using a similarity measure or a distance metric to connect the two entities is a reasonable approach.

Another frequently encountered theme is the analysis of a *binary relationship*. For example, in a retail use case, we may be interested in understanding the relationship between *customer* and *product*. In a health care use case, we may be interested in understanding the relationship between *patient* and *treatment*. In a manufacturing use case, we may be interested in understanding the relationship between *machine* and *operator*. In such cases, we may want to use a graph structure that is centered around the two entities of interest. This is also a heterogeneous graph structure where the nodes are different types of entities. Specifically, this type of graph is a *bipartite* graph. Understanding how a pivotal entity interacts with another entity is a common analysis task in enterprise use cases. See [10] for a detailed discussion of theory and applications of bipartite graphs. Personalization is also common motivation for this type of analysis. See [11] for a detailed discussion of personalization.

Finally, the third common theme we counter is a small set of connected entities and relationships. The analysis tasks in this case are typically understanding the behavior of or trying to predict:

1. The property of an specific entity
2. The property of a relationship between two entities
3. The property of a collection of entities

For these problems, we can map the source entities and relationships to a graph structure that replicates the entity relationships in the source system. This is a heterogeneous graph structure where the nodes are different types of entities. The edges are the relationships between the entities. The tasks described above correspond to *node classification*, *link prediction* and *graph classification* tasks in the graph learning literature. See [19] for a detailed discussion. For a detailed discussion of graph representation learning, please see [9].

The analysis themes discussed above are by no means exhaustive. These are just the common themes that we have observed in enterprise use cases. Graphs have been used in many other ways to solve a variety of problems in engineering and science. For a discussion of other applications, for example to science and engineering, please see [4].

The analysis for heterogeneity is typically different for each of these three themes. Please see the examples section for some illustrative techniques. *Graphs do make it easier to account for heterogeneity in the analysis. This is primarily because most analysis methods consider a neighborhood of a node to predict a property or an edge connected to the node. The neighborhood of a node captures the heterogeneity information related to the node.* There is some upfront work to create the graph, but once the graph is created, the huge body of work on graph analysis can be brought to bear on the problem.

4 Knowledge Graph Construction

A review of the document so far will show that there are several steps in the production of a graph from relational data where modeler has to make decisions and choices. The modeler has to understand the use case, the data, the sources of heterogeneity in the data and the type of analysis that is required for the use case. The modeler also has to understand the different types of graph structures and their suitability for different types of analysis.

Knowledge graphs make it possible to capture the decisions and choices made by the modeler in a structured way. *Knowledge Management for Data Science (KMDS)* is a framework for capturing the decisions and choices made by the modeler in a structured way. The framework is implemented in Python and is available as an open source project on GitHub [13]. This framework needs work to integrate it with LLM tools.

However, the integration of KMDS with the steps outlined in this document is a work in progress. KMDS is mentioned here to highlight the importance of capturing the decisions and choices made by the modeler in a structured way.

5 Examples of Implementation

5.1 Homogeneous Graph Construction for Feature Engineering

Graphs can be used to develop features for a machine learning model. In this example, we consider the problem of developing a representation for good borrowers in the SBA 7a loans dataset. This representation can be used for tasks such as developing a taxonomy of good borrowers. Understanding the heterogeneity of good borrowers can inform the design of new loan products. It can also help identify good borrowers that are similar to bad borrowers. This can help identify potential risks in the loan portfolio. In this example, we represent good borrowers as a graph and then develop a representation for this graph using a spectral embedding technique. The representation can then be used in a machine learning model. Please see this document for details.

5.2 Homogeneous Graph Construction for Representation Learning

Graphs can be used to develop a representation for a set of entities. In this example, we consider the problem of developing a representation for good borrowers in the SBA 7a loans dataset. This representation can be used for tasks such as developing a taxonomy of good borrowers. Understanding the heterogeneity of good borrowers can inform the design of new loan products. It can also help identify good borrowers that are similar to bad borrowers. This can help identify potential risks in the loan portfolio. In this example, we represent good borrowers as a graph and then develop a representation for this graph using a spectral embedding technique. The representation can then be used in a machine learning model. Please see this document for details. Note that this example is different from the previous example in that the representation is used for unsupervised learning. There is a single entity of interest, the good borrower. However, a similar approach can be used when multiple entities of interest are present and collaborate to accomplish a task. We will explore this in subsequent examples.

5.3 Bipartite Graph Construction from Relational Data

Bipartite graphs can be used to represent relationships between two types of entities. The page shows an example of using descriptive analytics to understand important characteristics of shopping behavior of customers in a retail setting. One part of the analysis examines the shopping behavior of customers in Sao Paulo, Brazil. A bipartite graph is constructed where the bipartite vertex sets ar the cities in Sau Paulo and the products purchased by customers in these cities. The edges represent the purchase of a product by a customer in a city. Please see this document for of the analysis. The details of the bipartite graph construction and the subsequent analysis are provided in this document.

5.4 Heterogeneous Graph Construction from Relational Data

The third common theme we counter is a small set of connected entities and relationships. As part of the input to the analysis, we have the entities and the relationships between the entities. We treat these entities and relationships as a heterogeneous graph. The analysis tasks in this case are typically are:

1. Predict the property of an specific entity, *node classification*
2. Predict the relationship between two entities, *link prediction*
3. Predict the label associated with a new set of related entities, *graph classification*

@inproceedingsschlichtkrull2018modeling, @bookgetoor2007introduction, A complete example of heterogeneous graph construction and analysis for node classification is provided in this article[16]. For guidelines and theoretical background for heterogeneous graph construction from relational data, please see [17]. For a comprehensive introduction to learning graphs from relational data, please see [8].

References

[1] Leo Breiman et al. *Classification and regression trees*. Chapman and Hall/CRC, 2017.

[2] Peter Pin-Shan Chen. "The entity-relationship model—toward a unified view of data". In: *ACM transactions on database systems (TODS)* 1.1 (1976), pp. 9–36.

[3] Edgar F Codd. "A relational model of data for large shared data banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387.

[4] Diane J Cook and Lawrence B Holder. *Mining graph data*. John Wiley & Sons, 2006.

[5] Ramez Elmasri and Shamkant B Navathe. *Fundamentals of database systems seventh edition*. pearson, 2016.

[6] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=fMHZt4UStUg&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=1.

[7] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=eI-hmySej_w&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=2.

[8] Lise Getoor and Ben Taskar. *Introduction to statistical relational learning*. MIT press, 2007.

[9] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.

[10] Jérôme Kunegis. "Exploiting the structure of bipartite graphs for algebraic and spectral graph theory applications". In: *Internet Mathematics* 11.3 (2015), pp. 201–321.

[11] Julian McAuley. *Personalized machine learning*. Cambridge University Press, 2022.

[12] Dorian Pyle. *Data preparation for data mining*. morgan kaufmann, 1999.

[13] Rajiv Sambasivan. *GitHub - rajivsam/KMDS: Source for KMDS — github.com*. https://github.com/rajivsam/kmds. [Accessed 06-09-2025].

[14] Rajiv Sambasivan. *Understanding Coffee Prices*. https://rajivsam.github.io/r2ds-blog/posts/markov_analysis_coffee_prices/. Accessed: 2025-06-21. 2025.

[15] Rajiv Sambasivan and Sourish Das. "Classification and regression using augmented trees". In: *International Journal of Data Science and Analytics* 7.4 (2019), pp. 259–276.

[16] Jörg Schad, Rajiv Sambasivan, and Christopher Woodward. "Predicting help desk ticket reassignments with graph convolutional networks". In: *Machine Learning with Applications* 7 (2022), p. 100237. ISSN: 2666-8270. DOI: https://doi.org/10.1016/j.mlwa.2021.100237. URL: https://www.sciencedirect.com/science/article/pii/S2666827021001195.

[17] Michael Schlichtkrull et al. "Modeling relational data with graph convolutional networks". In: *European semantic web conference*. Springer. 2018, pp. 593–607.

[18] Chin-Chia Michael Yeh et al. "Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets". In: *2016 IEEE 16th international conference on data mining (ICDM)*. Ieee. 2016, pp. 1317–1322.

[19] Jie Zhou et al. "Graph neural networks: A review of methods and applications". In: *AI open* 1 (2020), pp. 57–81.